

TP3 - Statistique avec R - Les fonctions

OBJECTIFS DU TP :

Le but de ce TP est s'initier à la programmation avec R. Nous verrons au passage comment éviter de faire des boucles.

Travaillez soit sur le [cluster R](#) soit sur le [Datalab/Onyxia](#). Vous pouvez aussi librement choisir de coder sur un document Quarto `.qmd` (au sein de *chunks* donc) ou sur un script `.R` classique.

! Pour les tuteurs :

Les étudiants ont créé un Rprojet `Stats_avec_R` au TP1, associé à leur repo Git personnel. Incitez-les à charger ce repo (via **File > New Project... > Version Control > Git** ou bien en suivant la procédure du TP1 : on colle le lien [https du repo](#) à l'ouverture du service **RStudio**, le token étant déjà renseigné).

Il est impératif que les étudiants sachent créer un RProjet "classique" (via **File > New Project > New Directory > New Project**) pour le TP noté. Ainsi, nous leur imposerons d'envoyer sur Moodle un `.zip` contenant les données + le script (bien plus simple pour vos corrections).

Commencer par charger votre Rprojet `Stats_avec_R` créé au TP1, puis affecter un nouveau dossier "TP3 fonctions". Pour importer vos données, cliquez sur le bouton **Upload** situé dans l'onglet **Files**, en dessous de votre environnement.

```
# Chargement des packages nécessaires
```

```
library(tidyverse)
```

Comment coder une fonction

Le langage R est un langage évolutif. Beaucoup de fonctions ont été créées (puis diffusées par le biais des packages). Vous serez certainement amenés à créer des fonctions pour vos besoins professionnels et, peut-être même, à créer des packages.

La syntaxe de création d'une fonction est la suivante :

```
ma_fonction<-function(argument1, argument2, autant_qu_on_veut){  
  Expression1  
  Expression2  
  ExpressionN  
}
```

Il est possible de créer des variables (et même des fonctions) au sein des fonctions. **Dans ce cas, ces variables (ou autre objet ne seront pas stockés dans votre environnement, sauf si vous les mettez en retour de fonction) :**

```
affiche_ma_valeur<-function(x){  
  print(paste("ca ne sert à rien mais affiche la valeur de x :",x))  
  z<-3  
}  
affiche_ma_valeur(4)  
# z n'a pas été enregistré dans l'environnement, car non appelé
```

```
# 2 façons pour qu'un objet soit stocké dans l'environnement:
# avec le return :
affiche_ma_valeur2<-function(x){
  print(paste("ca ne sert à rien mais affiche la valeur de x :",x))
  z<-3
  return(z)
}
z1 <- affiche_ma_valeur2(5)

# Ou (à éviter) avec <-
affiche_ma_valeur3<-function(x){
  print(paste("ca ne sert à rien mais affiche la valeur de x :",x))
  z <- 3
}
affiche_ma_valeur3(5)
```

Il est important de penser à mettre des tests, warnings lorsqu'on programme, dès lors qu'on souhaite transmettre ses fonctions. Le CRAN exige d'ailleurs toute une série de tests aux nouveaux packages avant d'accepter leur diffusion.

```
# On peut mettre des stop si une fonction ne respecte pas certains criteres

carre<-function(x){
  stopifnot(is.numeric(x))
  return(x^2)
}
```

Les boucles

En R, les boucles peuvent prendre du temps. R est un langage vectorisé : le plus souvent les fonctions/tests s'appliquent à tous les éléments du vecteur sans avoir à boucler sur ces éléments. En présence de données de grande dimension, les temps de traitement sont beaucoup plus rapides en travaillant directement sur un vecteur.

Vous travaillerez forcément un jour ou l'autre sur des bases de données très volumineuses. Gagner en temps de traitement devient alors un point important. Le langage R offre une famille de fonctions pour éviter de faire des boucles **for** notamment. Vous pouvez avoir une présentation rapide [en cliquant ici](#)

De manière générale, formez-vous (au-delà de ce cours)! Plusieurs packages existent pour éviter les boucles généralement pour simplifier l'utilisation des fonctions **apply** (type d'output stable). Vous pouvez notamment regarder le package **purrr**.

Vous verrez également d'autres méthodes (en 2e année) pour réduire les temps de traitement : le calcul parallèle.

Exercice 1

Dans cet exercice, on souhaite créer une fonction qui repère les valeurs manquantes et fait ensuite de l'imputation par la moyenne.

Première partie

1. Commencer par simuler un jeu de données. Pour cela, vous générerez 5 vecteurs :
 - `var1` : vecteur composé des 26 lettres de l'alphabet (vecteur `LETTERS`),
 - `var2` : vecteur composé de 26 variables aléatoires iid suivant une loi de Poisson de paramètre $\lambda = 3$,
 - `var3` : vecteur de 26 variables aléatoires iid suivant une loi Uniforme $U[50,120]$,
 - `var4` : vecteur de 26 variables aléatoires iid suivant une loi normale $N(50,15)$
 - `var5` : vecteur de 26 variables aléatoires iid suivant une loi binomiale $(100,0.8)$

Vous assemblerez ces vecteurs sous forme de `data.frame`. Votre objet `data.frame` s'appellera "`data`".

2. Nous allons maintenant générer des valeurs manquantes sur le vecteur `var1`. Pour cela, lancer le code suivant :

```
# Copie du dataframe
data_na <- data
# Nombre de NA généré (nombre aléatoire allant de 1 à 5)
n <- sample(1:5,1)
# Génération des NA sur la variable var1
data_na[sample(1:nrow(data_na),n), "var1"] <- NA
```

3. Appliquer le code précédent sur l'ensemble de vos variables. Vous utiliserez une boucle `for()`.
4. Adapter le code de la question 2 pour créer une fonction `genere_na()` qui génère aléatoirement des valeurs manquantes sur un vecteur. Cette fonction retournera un objet de type vecteur. La fonction prendra 2 arguments :
 - `x` = un vecteur sur lequel on souhaite générer des valeurs manquantes
 - `n` = le nombre de NA maximum,
5. Testez votre fonction, en refaisant la question 3 (utilisation de la fonction avec boucle `for()`). On fixera à 10 le paramètre de valeurs manquantes maximum. Au lieu de "`data_na`", votre objet en sortie s'appellera "`data_na_q5`".
- 6.a Faire une boucle n'est souvent pas nécessaire. Faire la question précédente en utilisant `dplyr` et sa fonction `mutate_all` appliquée à la fonction `genere_na()`. Votre objet en sortie s'appellera `data_na_q6a`.
- 6.b Faire maintenant avec la fonction `apply()`. Votre objet en sortie s'appellera "`data_na_q6b`".
7. Comparer les temps d'exécution entre la boucle `for()` (question 5) et les méthodes de la question 6. Pour cela, vous ferez précéder les traitements des questions 5 et 6 de la fonction `system.time()`. Quelle syntaxe est la plus rapide ?
8. Voyez-vous d'autres différences que le temps d'exécution ?

Deuxième partie

9. Créer rapidement une fonction `impute_na()` qui teste si un vecteur contient des valeurs manquantes et, si c'est le cas, remplace les valeurs manquantes par la moyenne. Tester la fonction sur le vecteur `c(1,NA,3)`.
10. Evidemment cette fonction ne peut s'appliquer qu'à des vecteurs numériques. Intégrer un test pour s'assurer que le vecteur est bien numérique sinon la fonction s'arrête (fonction `stop()`). Votre fonction contiendra aussi des avertissements à destination de l'utilisateur (fonction `warning()`). Par exemple, si le vecteur ne contient pas de valeurs manquantes. Tester la fonction sur le vecteur `c(1,2,3)`.
11. Ajouter un squelette à votre fonction pour la commenter. Pour cela appuyer sur `CTRL+MAJ+ALT+R` et complétez. Vous devez notamment renseigner la nature des paramètres et expliquer ce que fait la fonction.
12. Appliquer cette fonction sur le jeu de données "`data_na_q5`" : il ne devra plus rester de valeurs manquantes parmi les variables numériques.
13. Faites la question précédente d'une autre façon : utilisez `dplyr` et la fonction `mutate_if()`, remplacez les valeurs manquantes par la moyenne.

14. Modifier la fonction `impute_na()` pour intégrer un paramètre “`methode`”. Ce paramètre pourra prendre les valeurs suivantes “moyenne” ou “mediane”. Selon la méthode choisie, l’imputation sera différente. Par défaut, l’argument `methode` vaudra “moyenne”.
15. Reprenez la fonction créée à la question 4 et y intégrer des tests. Vous devez y intégrer au moins un test bloquant (stop) et un message d’avertissement (warning). À vous de trouver des idées !

Exercice 2

Recupérer la liste suivante. Il s’agit des films dans lesquels chaque personnage de starwars apparaissait.

```
library(dplyr)
films <- starwars$films

class(films[[1]]) # ou typeof()
```

1. Plutôt que d’avoir les titres des films, on souhaite avoir le nombre de films dans lequel le personnage apparaît. Comment vous y prendriez-vous ?
2. Les fonctions `lapply()` et `sapply()` permettent d’appliquer une fonction à chaque élément d’une liste ou d’un vecteur. Utiliser la fonction `lapply()` pour effectuer ce travail.
3. Utiliser maintenant la fonction `sapply()`. Quelle différence voyez-vous ?
4. On va maintenant travailler sur le jeu de données `starwars` complet. Quels sont les personnages qui ont été dans le film “A New Hope” ? Vous pouvez utiliser la fonction `stringr::str_detect()`.
5. Calculer la taille moyenne selon l’espèce. Vous utiliserez la fonction `tapply()` qui permet les boucles de type “group by”.

Exercice 3

Vous avez examen aujourd’hui, un QCM. Vous n’avez pas révisé mais vous vous dites le fameux “[sur un malentendu ça peut marcher](#)”. Le QCM comporte 20 questions. Chaque question comporte 4 propositions avec une seule réponse possible. Faites une simulation de 10 000 tentatives pour estimer la probabilité de valider la matière (avoir au moins 10).